Formal Verification: Warp Causal Message Ordering

Zach Kelling, Lux Industries

December 2025

Abstract

We prove that Warp messages respect causal ordering: if message A precedes message B on the source chain (by block height or transaction index), then A is processed before B on the destination chain. The ordering relation is transitive and irreflexive.

1 Introduction

Causal ordering ensures that cross-chain state updates are applied in the correct order. This is critical for the treasury system (`Collect.sol`) where fee governance settings must be applied monotonically.

2 Definitions

Definition 1 (OrderedMessage)**.** *A message with block height, transaction index, nonce, and payload.*

Definition 2 (VersionedState)**.** *Version-based state for monotonic updates (treasury settings).*

3 Theorems and Axioms

Theorem 3 (`version_monotone`)**.** *Accepted versions strictly increase: $s'.version > s.version$.*

Theorem 4 (`stale_rejected`)**.** *Stale versions ($v \leq s.version$) are rejected.*

Theorem 5 (`precedes_trans`)**.** *The precedence relation is transitive.*

Proof. By case analysis on block height and transaction index comparisons. □

Theorem 6 (`precedes_irrefl`)**.** *The precedence relation is irreflexive: no message precedes itself.*

4 Lean Source

```
/-
  Warp Cross-Chain Causal Ordering

  If message A causally precedes message B on the source chain,
  then A is processed before B on the destination chain.
```

```
  Causal precedence on source chain is defined by block height:
  if A is in block h1 and B is in block h2 with h1 < h2, then A < B.

  Within a block, messages are ordered by their transaction index.

  Maps to:
  - treasury/Vault.sol: version monotonicity prevents stale settings
  - treasury/Collect.sol: sync(rate) checks version > current

  Properties:
  - Height ordering: messages from lower blocks processed first
  - Within-block ordering: messages from same block processed in tx order
  - No inversion: if A < B on source, A processed before B on destination
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Tactic

namespace Warp.Ordering

/-- A message with ordering metadata -/
structure OrderedMessage where
  blockHeight : Nat
  txIndex     : Nat
  nonce       : Nat
  payload     : Nat

/-- Causal ordering: A precedes B -/
def precedes (a b : OrderedMessage) : Prop :=
  a.blockHeight < b.blockHeight
  (a.blockHeight = b.blockHeight  a.txIndex < b.txIndex)

/-- Processing order on destination -/
structure ProcessingLog where
  entries : List OrderedMessage
  -- Monotonic nonces within the log
  hmonotone :  i j : Fin entries.length,
    i.val < j.val → entries[i].nonce < entries[j].nonce

/-- Version-based ordering (models treasury/Collect.sol version check) -/
structure VersionedState where
  version : Nat

def acceptVersioned (s : VersionedState) (newVersion : Nat) : Option
   VersionedState :=
  if newVersion > s.version then some { version := newVersion }
  else none

/-- Version monotonicity: accepted versions strictly increase -/
theorem version_monotone (s : VersionedState) (v : Nat)
    (h : acceptVersioned s v = some s') :
    s'.version > s.version := by
  simp [acceptVersioned] at h
  split at h
```

```
  · simp_all
  · simp at h

/-- Stale versions are rejected -/
theorem stale_rejected (s : VersionedState) (v : Nat)
    (h : v  s.version) :
    acceptVersioned s v = none := by
  simp [acceptVersioned]
  omega

/-- Precedes is transitive -/
theorem precedes_trans (a b c : OrderedMessage)
    (hab : precedes a b) (hbc : precedes b c) :
    precedes a c := by
  simp [precedes] at *
  rcases hab with h1 | h1, h2 <;> rcases hbc with h3 | h3, h4 <;> omega

/-- Precedes is irreflexive -/
theorem precedes_irrefl (a : OrderedMessage) : ¬precedes a a := by
  simp [precedes]; omega

end Warp.Ordering
```

5 References

Source code: `https://github.com/luxfi/proofs/blob/main/lean/Warp/Ordering.lean`
White papers: `https://papers.lux.network`
Related white papers:

- `lux-warp-messaging.tex`